

# C-Bounded Powersort's time complexity and correctness analysis

Théo Araújo Magalhães, Victor Almeida Campos

April 5, 2025

## 1 Definitions:

*Clarification:* Definitions with a \* at the end come from Munro and Wild's papers. This is a 'to-do', I will properly cite them later.

**Runs:** Contiguous regions of the input that are already in sorted order. Let  $R_0, \dots, R_{r-1}$  be the runs in the input,  $L_i$  denotes the length of  $R_i$ .\*

**Run-adaptive mergesort:** Any stable run-adaptive mergesort can be described as a merge tree  $T$ : an (ordered rooted) tree with leaves  $R_0, \dots, R_{r-1}$  (appearing in that order from left to right). Each internal node  $v$  of  $T$  corresponds to the result of merging its children.\*

**Merge costs:**  $M(v)$  is the total length of the resulting merge corresponding to the internal node  $v$ . The total merge cost of  $T$  is then:

$$M(T) = \sum_{v \in T} M(v). \quad (1)$$

**Run boundaries:** Denoted by  $B_i$ , it is the boundary between the  $(i - 1)$ st and  $i$ th run ( $i=1, \dots, r - 1$ ). In binary merge trees, there is a one-to-one correspondence between the  $B_i$  and the (inner/non-leaf) merge-tree nodes.\*

**Node depth:** Let  $d_i$  be the depth of leaf  $R_i$  in  $T$ , where depth is the number of edges on the path to the root. Every element in run  $R_i$  is counted exactly  $d_i$  times in  $\sum_v M(v)$ , so:

$$M = \sum_{i=0}^{r-1} d_i \cdot L_i \quad (2)$$

**Node power:** Let  $\alpha_0, \dots, \alpha_m, \sum \alpha_j = 1$  be leaf probabilities. For  $1 \leq j \leq m$ , let  $j$  be the internal node separating the  $(j - 1)$ st and  $j$ th leaf. The power of (the split at) node  $j$  is:

$$P_j = \min\{\ell \in \mathbb{N} : \lfloor \frac{a}{2^{-\ell}} \rfloor < \lfloor \frac{b}{2^{-\ell}} \rfloor\}, \text{ where } a = \sum_{i=0}^{j-1} \alpha_i - \frac{1}{2} \alpha_{j-1}, b = \sum_{i=0}^{j-1} \alpha_i + \frac{1}{2} \alpha_j. \quad (3)$$

( $P_j$  is the index of the first bit where the (binary) fractional parts of  $a$  and  $b$  differ.)

**Powersort:** The intuition behind Powersort is to imagine a complete binary tree  $T_v$  sitting on top of the array, where each element  $A[i]$  of the input corresponds to

a leaf of  $T_v$ .  $T_v$  corresponds to the merge tree of a non-adaptive two-way Mergesort (starting with individual elements). Now each run boundary  $B_j$  also corresponds to an (inner) node  $w$  of this (virtual)  $k$ -ary tree  $T_v$ , namely the lowest common ancestor of the two leafs adjacent to  $B_j$ . \*[ We need to re-explain this part better so that we can talk about the concept of a “Powersort merge tree” or something of the sorts. Helps with the proof of our main proposition]

**Forgetful K-Stack:** A forgetful stack is a stack that forgets its  $k$ -th element if it already has  $k$  elements in it and a new one is added.

**C-Bounded Powersort:** The Powersort algorithm using a Forgetful C-Stack instead of the traditional stack with size  $\log n + 1$ .

**Processing a subtree of height  $h$ :** Let  $T$  be a merge tree,  $T'$  one of its subtrees of size  $h$  and  $v$  the root of  $T'$ . Processing  $T'$  is executing all the merges necessary to fill node  $v$ , given that it has length  $M(v)$ .

## 2 Proofs:

**Proposition:** Let  $T$  be a Powersort merge tree of height  $H$ . One pass of the C-Bounded Powersort algorithm is enough to process all subtrees of  $T$  with height of  $C-1$ .

*Proof by induction:*

**Base case -  $T$  of height 2:**

Let  $v$  be the root of  $T$  and  $v_1$  and  $v_2$  its two children. Therefore,  $T$  has two subtrees,  $T'$  and  $T''$ , both of height one. For the left subtree  $T'$ ,  $v_1$  is the root and it has only two leaf children: the runs  $R_1$  and  $R_2$ . Given that  $h = 1$ , a Forgetful 2-Stack will work with 2-Bounded Powersort as follows:

$R_1$  will be found and added to the stack, then, given that there is nothing to compare  $R_1$  with, the algorithm will search for the adjacent run,  $R_2$ . After finding it, the algorithm will calculate the power between  $R_1$  and  $R_2$ , which is equal to the height of  $v_1$ . Given that we only have one power calculated, this will lead to no merges and  $R_2$  will be added to our stack. Afterwards, the algorithm will proceed to the right  $T''$  subtree, which is rooted in  $v_2$ . The algorithm will find  $R_3$  and, then, calculate the power between  $R_2$  and  $R_3$ . This will result in the height of  $v$ , which is smaller than that of  $v_1$ , our previously calculated power, leading to the merge of runs  $R_1$  and  $R_2$  into run  $R_{12}$ .

After this, our Forgetful 2-Stack will contain  $R_{12}$  and  $R_3$ . Then, the algorithm will find run  $R_4$ . Calculating the power between  $R_3$  and  $R_4$  leads to height  $v_2$ , which is bigger than that of  $v$ , our previously calculated power, meaning the algorithm will not merge any runs and will add  $R_4$  to the Forgetful 2-Stack. This will cause the stack to “forget” about  $R_{12}$ , meaning it will only contain  $R_3$  and  $R_4$ . Once we find  $R_4$ , we will have reached the end of the array.

Following the logic of the algorithm, once the scan reaches the end of the array everything that still is in the Forgetful-Stack will be merged, thus, in this situation, a merge between  $R_3$  and  $R_4$  will happen, leading to the processing of both subtrees  $T'$  and  $T''$ . This concludes our base case.

**Hypothesis:** Suppose the proposition holds true for all subtrees of height  $h \leq k$  in a powersort merge tree.

**Inductive Step:** Let  $T$  be a Powersort merge tree of height  $H$ . We will prove that a  $(k+2)$ -Bounded Powersort processes all of  $T$ 's subtrees that have height of  $h = k+1$ . Let  $T'$  be one of those subtrees and  $v$  its root. We will analyze the sub-trees  $T_\ell$  and  $T_r$ , each rooted in the left and right children of  $v$ ,  $v_\ell$  being the left child and  $v_r$  the right child. The height of the  $T_\ell$  is, at most,  $k$ , therefore, we can apply our induction hypothesis and process  $T_\ell$ , thus,  $v_\ell$  will contain run  $R_\ell$ , which is the result of the merge of all runs in  $T_\ell$ . The processing of this sub-tree will happen once the most-left run of  $T_r$  is analyzed by the algorithm.<sup>1</sup> By the end of the processing of  $T_\ell$ , our Forgetful  $(k+2)$ -Stack will contain  $R_\ell$  and the most-left run of  $T_r$ . Given that, by hypothesis, the processing of  $T_r$  requires one pass of a  $(k+1)$ -Bounded Powersort<sup>2 3</sup>, and that our Forgetful  $(k+2)$ -Stack has exactly  $k$  spaces left, with one of them already being occupied by one run of  $T_r$ , the algorithm will be able to process  $T_r$ . Similarly to what happened during the processing of  $T_\ell$ , the calculation of the power between the right-most run of  $T_r$  (which is also the right-most run of  $T'$ ) and the next run, which is the left-most run of another subtree of  $T$  with height  $k+1$ , will result in the height of  $v - 1$ .<sup>4</sup> Given that all the powers calculated between runs in  $T'$  are greater than  $v-1$ , the algorithm will merge all the remaining runs in our Forgetful  $(k+2)$  stack, leading to the processing of  $T_r$  but also to the processing of  $T'$ , given that  $R_\ell$  will still be on the stack alongside all runs from  $T_r$ . Therefore, if the  $(k+2)$ -Bounded Powersort was able to process a generic  $T'$  subtree of  $T$  with height  $k+1$ , it is fair to assume that it will be able to further process all the remaining subtrees of same height by following the same logic as the one described for  $T'$ .  $\square$

---

<sup>1</sup>Once the power between the most-right run of  $T_\ell$  with most-left run of  $T_r$  is calculated, it will result in the height of  $v$ , which is smaller than the height of  $v_\ell$ , meaning the algorithm will merge everything left in our Forgetful  $k+2$ -Stack, for all powers calculated between runs in  $T_\ell$  is greater than  $v_\ell$

<sup>2</sup>Remember that  $T_\ell$  and  $T_r$  have, at max, height of  $k$

<sup>3</sup>Requiring a  $(k+1)$ -Bounded Powersort means the processing needs  $k+1$  free spaces in the Forgetful C-Stack, not a specific tuning of the algorithm.

<sup>4</sup>This is due to the fact that both runs are in adjacent subtrees:  $v$  is the left child of a node  $v_p$  and there is a node  $v'$  which is the right child of the same  $v_p$ , the first common ancestor between  $v$  and  $v'$  is  $v_p$ , therefore, the first common ancestor between runs in each subtree (the subtree rooted in  $v$  and the subtree rooted in  $v'$ ) is  $v_p$

**Proposition:** It takes C-Bounded Powersort at most  $\mathcal{O}(n \cdot \log n)$  element scans to order an array of size  $n$ .

*Proof.* Each pass through the array scans element  $i$ , with  $0 \leq i \leq n$ , at least once. Given that the algorithm will do  $\frac{\log n}{c-1}$  passes<sup>5</sup>,  $i$  will be read at least  $\frac{\log n}{c-1}$  times. Once the entire array is ordered,  $i$  will also have been read at least twice for each merge it was present in, i.e,  $2 \cdot d_i$ , with  $d_i$  being equal to the height of  $T$ . Therefore, the total number of scans is:

$$Scans \leq \sum_{i=0}^n 2 \cdot \log n + n \cdot \frac{\log n}{c-1} \quad (4)$$

$$Scans \leq n \log n \cdot (2 + \frac{1}{c-1}) \quad (5)$$

$$Scans \leq \mathcal{O}(n \cdot \log n) \quad (6)$$

□

---

<sup>5</sup>The height of a Powersort merge tree  $T$  is  $\log n$  and the C-Bounded Powersort algorithm starts by processing subtrees of  $T$  that have height less than or equal to  $c-1$ . This results in a smaller tree  $T'$  of height  $\log n - (c-1)$ . A second pass will, once again, process the subtrees of  $T'$  that have height less than or equal to  $c-1$ , resulting in a smaller tree  $T''$ . Therefore, if  $k$  is the amount of trees created by this process until the last tree is of height less than or equal to  $c-1$ , meaning it will only require one last pass of the algorithm to be processed,  $k \cdot (c-1) \leq \log n$

### 3 Pseudocodes:

#### TO-DOs:

1. Os comentários para cada algoritmo precisam estar de acordo com as definições usadas no primeiro esboço do artigo (powersort.tex) e precisam ser revisados para fazerem sentido, alguns ainda estão bem abstratos e/ou não descrevem corretamente os algoritmos.
2. Tanto para esse pdf quanto para o anterior, ou seja, para o artigo como um todo, o nome do professor Victor se enquadra como autor? Imagino que sim. Essa pergunta é meio boba.
3. Código para a estrutura de dados que iremos usar.
4. Reestruturar o vai e volta.

**Planejamento da estrutura:** Guardaremos o início da primeira run, seu fim e seu power com a run anterior (se houver), além disso, cada elemento seguinte será: o fim da run e o power da run com a anterior. run e não teremos um elemento sentinela. O início de uma run baseia-se no fim da anterior + 1. Assim que incluímos um elemento que estoura o tamanho da pilha, temos que esquecer o primeiro elemento e ajustar o novo primeiro elemento para também guardar o seu início. (cada pop ou insert decrementa ou aumenta o tamanho)

Como vamos fazer scans da esquerda para a direita e da direita para a esquerda, precisamos alternar entre somar um no fim de uma run para achar o começo da próxima e diminuir um. ← pensar nisso

**Comentário do professor:** “Só um detalhe de notação, usa aqui  $(A, s_2, n)$ .  $A[s_2]$  é o valor e não o índice.”

**Resposta:** Fiz desse jeito porque é a forma que eles escreveram no artigo, mas achei bem estranho também, depois vou mudar.

**Introduction:** We are very smart and we know things, this is the complaints we have for things that weren't done by us and how we would have made them better.

### 3.1 Powersort Versions:

#### 3.1.1 Standard Powersort

**Commentary on Wild and Munro's version:** A single pass algorithm that sorts the entire array.

---

**Algorithm 1** Powersort( $A[1..n]$ ) by Wild and Munro:

---

```
1:  $X :=$  stack of runs ▷ capacity  $\lfloor \lg n(n) \rfloor + 1$ 
2:  $P :=$  stack of powers ▷ capacity  $\lfloor \lg n(n) \rfloor + 1$ 
3:  $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$ 
4: while  $e_1 < n$  do
5:    $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n)$  ▷  $A[s_2..e_2]$  next run
6:    $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$  ▷  $P_j$  for node  $j$  between  $A[s_1..e_1]$  and  $A[s_2..e_2]$ 
7:   while  $P_{\text{top}}() > p$  do ▷ previous merge deeper in tree than current
8:      $P_{\text{pop}}()$  ▷  $\rightarrow$  merge and replace run  $A[s_1..e_1]$  by result
9:      $(s_1, e_1) := \text{Merge}(X_{\text{pop}}(), A[s_1..e_1])$ 
10:   $X_{\text{push}}(A[s_1, e_1]); P_{\text{push}}(p)$ 
11:   $s_1 := s_2; e_1 := e_2$ 
12: end while ▷ Now  $A[s_1..s_1]$  is the rightmost run
13: while  $\neg X_{\text{empty}}()$  do
14:    $(s_1, e_1) := \text{Merge}(X_{\text{pop}}(), A[s_1..e_1])$ 
```

---

### 3.1.2 First variation of C-Bounded Powersort (only goes right)

**Commentary on our first version:** This algorithm performs multiple passes on the array, processing all subtrees of height  $C-1$  with each pass. We remind the reader of how Powersort works: It is implicitly a complete binary tree of size  $\log n$ . Therefore, the max amount of passes needed to sort the array is:

$$\text{N}^\circ \text{ of scans} \leq \frac{\log n}{C-1} \quad (7)$$

---

**Algorithm 2** C-Bounded-Powersort 1( $A[1..n]$ ,  $C$ ):

---

```

1:  $X :=$  C-bounded stack of runs ▷ capacity C
2:  $P :=$  C-bounded stack of powers ▷ capacity C
3:  $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$ 
4: while  $e_1 < n$  do ▷ This while can be tweaked to use max amount of passes
5:   while  $e_1 < n$  do ▷ A single pass goes until the end of the array
6:      $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n)$ 
7:      $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$  ▷ Using our node power algorithm
8:     while  $P.\text{top}() > p$  do ▷ We follow the same merge policy
9:        $P.\text{pop}()$ 
10:     $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
11:     $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 
12:     $s_1 := s_2; e_1 := e_2$ 
13:  end while
14:  while  $\neg X.\text{empty}()$  do
15:     $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
16:    ▷ We're using a limited stack, so we need to check for another pass
17:     $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$  ▷ If  $e_1 = \text{end}$ , the array is sorted
18: end while

```

---

**After first modifications:**

### 3.1.3 Second variation of C-Bounded Powersort (goes left and right!)

**Commentary on our second version:** This works rather similarly to our first version. The main difference is the fact that, once we reach the end of the array, instead of restarting our scan from the beginning, we scan, starting from the end, to the left, until we reach the beginning of the array, and, thus, we repeat the scan-right process. This is slightly better than the first version, given that it takes advantage of runs still being in cache instead of restarting from the beginning.

**Comentário sobre  $X.size > 1$ :** Ao invés de termos uma stack  $X$  e  $P$ , teremos somente uma Stack guardando power e runs. Assim, caso mantenhemos uma run dentro da stack após o final de um scan, ela não estará com o 'power' correto quando o próximo scan iniciar. Isso significa que teríamos que dar um pop nessa pilha e tratá-la como vazia. Podemos fazer um  $s_1 \leftarrow top$ , visto que a run a ser achada por  $e_1 := ExtendRunSide$  depois de um scan vai ser sempre a run resultante de todos os merges finais (será?)

---

**Algorithm 3** C-Bounded-Powersort 1( $A[1..n]$ ,  $C$ ):

---

```

1:  $X := C$ -bounded stack of runs and powers
2:  $s_1 := 1; e_1 = ExtendRunRight(1, n)$ 
3: while true do
4:   while  $e_1 < n$  do
5:      $s_2 := e_1 + 1; e_2 := ExtendRunRight(s_2, n)$ 
6:      $p := NodePower(s_1, e_1, s_2, e_2, n)$ 
7:     while  $X.size > 1$  and  $X.top().power > p$  do
8:        $s_0 \leftarrow X.pop().boundary$ 
9:       Merge( $A, s_0, s_1, e_1$ )
10:       $s_1 \leftarrow s_0$ 
11:       $X.push(s_1, p)$ ;
12:       $s_1 := s_2; e_1 := e_2$ 
13:   end while
14:   while  $X.size > 1$  do
15:      $s_0 \leftarrow X.pop().boundary$ 
16:     Merge( $A, s_0, s_1, e_1$ )
17:      $s_1 \leftarrow s_0$ 
18:    $s_1 \leftarrow X.pop().boundary$ 
19:   swap( $s_1, e_1$ )  $\triangleright$  Simplification for  $s_1$  gets  $e_1$  and  $e_1$  gets  $s_1$ . This signals the
      start of the scan left.
20:    $\triangleright$  After merging everything that was on the stack, we will be left with
      a run at the end of the array. We will use this run as our first run in our scan left.
      This saves on doing an initial scan left to 'switch sides' and find the first run.
21:   if  $e_1 == 1$  then
22:     break  $\triangleright$  If we could extend until the beginning, our run is the entire array.
23:   while  $e_1 > 1$  do  $\triangleright$  We're scanning from  $n$  down to 1

```



---

```

24:    $s_2 := e_1 - 1; e_2 := \text{ExtendRunLeft}(s_2, 1)$ 
25:    $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$ 
26:   while  $X.\text{size} > 1$  and  $X.\text{top}() > p$  do
27:      $s_0 \leftarrow X.\text{pop}().\text{boundary}$ 
28:      $\text{Merge}(A, s_0, s_1, e_1)$ 
29:      $s_1 \leftarrow s_0$ 
30:      $X.\text{push}(s_1, p);$ 
31:      $s_1 := s_2; e_1 := e_2$ 
32:   end while
33:   while  $X.\text{size} > 1$  do
34:      $s_0 \leftarrow X.\text{pop}().\text{boundary}$ 
35:      $\text{Merge}(A, s_0, s_1, e_1)$ 
36:      $s_1 \leftarrow s_0$ 
37:    $s_1 \leftarrow X.\text{pop}().\text{boundary}$ 
38:    $\text{swap}(s_1, e_1)$   $\triangleright$  Switching to the scan right. The equivalent of what we did
    before scanning left.
39:   if  $e_1 == n$  then
40:     break

```

---

## 3.2 NodePower versions:

### 3.2.1 Standard Nodepower:

**Commentary on Wild and Munro's Node Power:** This is so silly and expensive!

---

**Algorithm 4**  $\text{NodePower}(s_1, e_1, s_2, e_2, n)$ :

---

```

1:  $n_1 := e_1 - s_1 + 1; n_2 := e_2 - s_2 + 1; \ell := 0$ 
2:  $a := (s_1 + n_1/2 - 1)/n; b := (s_2 + n_2/2 - 1)/n$ 
3: while  $\lfloor a \cdot 2^\ell \rfloor == \lfloor b \cdot 2^\ell \rfloor$  do  $\ell := \ell + 1$  end while do
4: return  $(\ell)$ 

```

---

### 3.2.2 Our Nodepower:

**Commentary on our Node Power:** Ours is cool and epic. (TO-DO: explanation of why it works)

---

**Algorithm 5**  $\text{NodePower}(s_1, e_1, s_2, e_2, n)$ :

---

```

1:  $\text{xor} := s_1 \wedge s_2$ 
2:  $\text{first\_bit} := \_\text{builtin\_clz}(\text{xor})$ 
3:  $\ell := \text{first\_b}$ 
4: return  $(\ell)$ 

```

---

### 3.3 The data structure:

**Introduction:** Wild and Munro's paper shows no specific code for their stack data structure, this is probably due to the fact that they simply use a standard stack of height  $\log n$ . Thus, we will have no 'counterpart' algorithm to analyze. Our algorithm implements a modified version of a stack which we call "C-Bounded Stack"/"Forgetful Stack".

#### 3.3.1 C-Bounded Stack:

**Commentary on our data structure:** When creating this data structure, we had to think of two particularities:

1. This is a stack of limited size.
2. This is a stack that has to consider scans going right and a scans going left.

In order to deal with the limited size, which is the 'forgetful' aspect of the data structure, we simply use circularity: we keep the index of where the next new element will be inserted in the case of a push and increment it. If we reach the end of the array, we assign the free space to be the first element and overwrite it in case of a push. As the stack fills, we also increment a variable for 'size', which corresponds to the amount of elements in the stack. Thus, we keep two variables, one for 'top' and one for 'size'. Other than that, each element in the stack consists of a run boundary and the power of this run with its predecessor.

**Run boundaries:** When the algorithm is scanning right, the boundary it keeps is the start of the run found. When the algorithm is scanning left, the boundary it keeps is the end of the run found.

---

**Algorithm 6** C-Bounded Stack(initial run  $s_1$ , size  $C$ ):

---

```
1: top := 1
2: size := 0
3: MaxSize := C
4: array[MaxSize]
5: func Pop(){
6:   if size > 1: then
7:     if top == 1: then
8:       top ← MaxSize
9:     else
10:      top--
11:    C-- }
12: func Push(run boundaryi, power k){
13:   array[top] ← (boundary1, k)
14:   top ← top%MaxHeight + 1
15:   if ( thenC ≠ MaxHeight:)
16:     C++
17:   }
17: func Top(){
18:   if C ≥ 1: then
```

```
19:   return array[(top == 1) ? MaxHeight : (top - 1)] }  
20: else  
21:   return null
```

---